

Mini-Jogo Concorrente com Servidor Erlang e Cliente Java/Processing

Programacao Concorrente - Grupo 17

Este relatorio descreve a arquitetura final, o protocolo de comunicacao e as decisoes de concorrencia usadas no mini-jogo. A implementacao segue uma abordagem simples: cada componente com estado partilhado e isolado num processo/objeto proprio, comunicacao por mensagens e cliente grafico separado da logica do servidor.

1. Objetivo e funcionalidades implementadas

O trabalho implementa um mini-jogo multiplayer em que varios utilizadores se registam, autenticam, entram numa fila de espera e sao agrupados em partidas de 3 ou 4 jogadores. O servidor permite ate 4 partidas simultaneas. Cada partida possui duracao controlada, mundo 2D, jogadores circulares, objetos comestiveis, objetos venenosos, movimento por tres teclas, colisoes, capturas, pontuacao e top em memoria.

Funcionalidades principais: registo, login, cancelamento de conta, fila de espera, top de pontuacoes enquanto o jogador espera, criacao de partidas, simulacao no servidor, atualizacao grafica no cliente e fim de jogo com vencedor ou empate.

2. Arquitetura geral

Componente	Responsabilidade
server.erl	Arranca os gestores, abre o socket TCP e aceita clientes.
connection_session.erl	Processo por cliente. Faz parsing, valida estado, encaminha pedidos e envia respostas.
auth_manager.erl	Guarda contas e sessoes online em memoria.
queue_manager.erl	Gere a fila de espera e seleciona grupos de 3 ou 4 jogadores.
game_supervisor.erl	Controla o limite de 4 partidas simultaneas e cria game_session.
game_session.erl	Liga cada partida ao motor de jogo e distribui snapshots pelos clientes.
game_engine.erl	Calcula fisica, objetos, colisoes, capturas e resultado final.
scoreboard_manager.erl	Mantem o top das melhores pontuacoes vencedoras desde o arranque.
Cliente Java	Gere socket, threads de leitura/escrita, input e estado partilhado.
Processing	Desenha login, espera, jogo e fim de partida.

3. Modelo de concorrencia no servidor

O servidor foi organizado segundo o modelo de atores de Erlang. Cada processo mantem o seu proprio estado local e comunica com os restantes por mensagens. Esta decisao evita memoria partilhada direta e reduz o risco de corridas sobre estruturas globais.

A connection_session e criada por cliente e apenas ela fala diretamente com o socket desse cliente. Os gestores globais - autenticacao, fila, supervisor e scoreboard - serializam naturalmente o acesso ao seu estado atraves do ciclo receive/loop. Assim, cada gestor comporta-se como um objeto concorrente simples, com estado encapsulado dentro do processo.

O game_supervisor impede que existam mais de 4 partidas simultaneas. Para evitar reservar indevidamente capacidade, a fila e o supervisor usam uma pequena fase de reserva/confirmacao: a fila pede criacao, o supervisor reserva capacidade se existir vaga, e a fila confirma quando remove os jogadores da fila.

4. Estados da sessao de cliente

Cada cliente passa por estados simples: connected, authenticated, waiting e in_game. A connection_session valida os comandos permitidos em cada estado. Por exemplo, login so e aceite em connected, join_queue so em authenticated, top_scores e leave_queue so em waiting, e input so em in_game.

Este controlo evita pedidos fora de ordem, como enviar movimento antes da partida ou fazer logout diretamente enquanto o jogador ainda esta na fila. Ao sair da sala de espera, o cliente envia primeiro `leave_queue`; so depois do servidor confirmar e voltar ao menu e feito logout.

5. Protocolo TCP

O protocolo é textual e orientado a linhas. Cada mensagem termina com newline, o que simplifica o uso de `get_tcp` no servidor com `{packet, line}` e `BufferedReader/PrintWriter` no cliente Java.

Exemplos de mensagens cliente-servidor:

```
create_account user pass
login user pass
join_queue
leave_queue
top_scores
input left down
input left up
input right down
input forward down
```

Exemplos de mensagens servidor-cliente:

```
login_ok user
join_queue_ok
top_scores nome:pontuacao,nome:pontuacao
game_started 1
game_config 800 600 50
game_state_begin tick
player username x y radius angle captures me|other
food id x y radius
poison id x y radius
game_state_end
game_over winner username captures
```

No servidor, as teclas recebidas em inglês pelo protocolo são traduzidas para os atores usados pelo motor: `left` -> esquerda, `right` -> direita e `forward` -> frente.

6. Simulação do jogo

A simulação é executada no servidor, não no cliente. O motor de jogo mantém jogadores, objetos comestíveis, objetos venenosos, tempo final e tick atual. A cada tick, aplica as teclas ativas, atualiza posições, limita bordas, trata venenos, capturas de comestíveis, capturas entre jogadores e reposição de objetos.

O raio dos avatares é calculado a partir da massa, usando a relação $\text{area} = \text{massa}$. Assim, como $\text{area} = \pi * r^2$, o raio é $\sqrt{\text{massa}/\pi}$. Esta fórmula é usada tanto no servidor para colisões como no cliente para representar visualmente o tamanho dos objetos.

Quando um jogador toca num objeto venenoso, perde massa até ao mínimo definido. Quando cobre completamente um objeto comestível menor, ganha a massa desse objeto. Quando cobre completamente outro jogador menor, ganha um quarto da massa do jogador capturado, aumenta a sua pontuação de capturas e o jogador capturado reaparece noutra posição.

No fim da partida, vence quem tiver maior número de capturas. Se houver empate no melhor número de capturas, a partida é ignorada no scoreboard.

7. Cliente Java e Processing

O cliente Java separa rede, input e estado. A thread de leitura recebe linhas do servidor e atualiza o `GameState`. A thread de escrita envia mensagens através de uma fila, evitando bloquear a interface gráfica. O `GameState` usa `ReentrantReadWriteLock` para permitir leituras seguras pelo `draw` do Processing enquanto a thread de rede atualiza dados.

O Processing apenas desenha e encaminha eventos. Existem ecrã de login/registo, sala de espera com top, ecrã de jogo e ecrã de fim de partida. O jogador local é desenhado com borda azul e os outros com borda vermelha; objetos comestíveis são verdes e venenosos vermelhos.

8. Decisões finais de integração

A entrega final coloca o motor de jogo dentro da pasta do servidor, para evitar erros de módulo não encontrado. A pasta Processing foi unificada e contém o `client.jar` correto. Também foi corrigida a zona clicável do ecrã de login e o fluxo de saída da fila.

O top guarda a melhor pontuação vencedora de cada jogador, de acordo com o número de capturas numa partida ganha. Isto mantém a implementação simples e respeita a ideia de mostrar pontuações de vencedores

desde que o servidor arrancou.

9. Como executar

Servidor: entrar em `src/servidor_erlang`, abrir `erl`, compilar os modulos com `c(nome_modulo)`. e executar `server:start()`. O servidor escuta na porta 12345.

Cliente: abrir `src/cliente_processing/MiniJogo` no Processing IDE e executar o sketch. Para formar partida, devem ser abertos 3 ou 4 clientes, cada um com uma conta diferente.

10. Conclusao

A solucao final privilegia simplicidade, separacao clara de responsabilidades e uso direto dos mecanismos estudados: processos Erlang, mensagens, sockets TCP, estado encapsulado por processo, threads Java para nao bloquear a interface e locks de leitura/escrita no estado compartilhado do cliente.